What Keeps a Generative Recommender Relevant? An Empirical Study of Reinforcement Fine-Tuning Against a Learned Reward Model

Daniil Antonov
ITMO University, Saint Petersburg, Russia
October 2026

Abstract. A recommender that generates its list item by item is a policy, so reinforcement learning (RL) can fine-tune it towards feedback that next-item training does not see. We study the setting with the fewest safeguards. PPO fine-tunes a GPT-2 recommender to raise the share of each item that users consume, as predicted by a second GPT-2, the reward model. The only term that asks for relevance is a KL penalty towards the pretrained recommender, the *anchor*. Because every reward gain is itself a prediction, we check it with signals the policy cannot influence: a reward model trained on other users, the logged responses, and the same lists scored for another user. We run on KION (video) and Yambda-50M (music), with three seeds for every central comparison. On KION, RL raises the predicted share by 8.3% for a 1.8% loss of NDCG@10 at the anchor weight chosen on validation, and by 24% for an 11.5% loss at a weaker one. What RL learns is mostly one re-weighting of the catalogue, the same for every user, towards the titles that viewers finish most often. This re-weighting is most of the reward's own optimum. RL settings therefore only move the policy along a frontier between relevance and reward, and the closed-form optimum, computed at inference, reaches the same frontier without training. The frontier itself moves only when training draws on the log or the list format changes. Reward-weighted fine-tuning on the logged share relative to what is typical for each item needs no reward model. Over the same extra epoch of plain training, it raises users' logged response beyond what is typical for each item by 21% and personalisation by 19%. Predicting each list position from the real history is 6% more relevant than generating the list.

1 Introduction

Generative sequential recommenders [1], [2], [3], [4], [5] are trained to continue logged sequences. But the log records what users did, not what served them: 36% of music listens in Yambda stop before half of the track, and 52% of KION titles are abandoned before half of their length. A recommender that generates a list item by item is a policy, so RL can move it towards what will be received well [6], [7], [8], [9]. How safe this is depends on where the reward comes from. Petrov and Macdonald [8] keep relevance inside the reward. Matveev et al. [9] learn from logged feedback and use a learned feedback model to evaluate and select recommendations at inference time. Letting a learned model of user feedback *be* the training reward is the option with the fewest protections. A policy optimised against a learned model drifts to where the model is wrong [10], [11], and in recommendation this means popularity and filter bubbles [12], [13].

We study this option in its barest form. A GPT-2 [14] recommender is fine-tuned to raise the consumed share that a second GPT-2 predicts. A KL penalty towards the pretrained recommender, the *anchor*, is the only term that asks for relevance. Since a model reports every reward gain in this setting, the study rests on instruments that the optimised model cannot influence, and on three seeds for every central comparison. Beyond measuring the trade-off, we ask why it arises and what moves it. For this we borrow probes from the post-training of language models: RL restricted to one score offset per item or to rank-1 updates; the policy's change split into a per-item part and the rest; the closed-form optimum computed at inference, with and without a learned look-ahead; learning from logged responses alone; and lists predicted position by position from the real history.

The study answers five questions about RL fine-tuning of a next-item recommender. Table 1 answers each in brief, Section 5 gives the evidence in one subsection per question, and Table 2 defines the terms that recur.

2 Related Work

Next-item models [1], [15], [16] run autoregressively as generative recommenders over items [2] or semantic identifiers [3], up to industrial scale [4], [5]. RL enters recommendation through off-policy policy gradients [6], slate values [7] and RL heads [17], and learned simulators replace the user [18], [19]. A policy optimised against a learned model exploits its errors [10], [11], [20], [21]. An ensemble of reward models does not stop this, because models trained alike share their errors [22]. Penalising divergence from a pretrained model is KL-

Table 1: The five questions and their answers, on KION unless Yambda is named.

Question	Answer
Q1. How does the model's behaviour change after RL fine-tuning?	The predicted reward rises (+24% on KION), and a reward model trained on other users agrees. But what RL learns is mostly one re-weighting of the catalogue, the same for every user, towards titles that everybody finishes. That re-weighting is most of the reward's own optimum.
Q2. Is relevance preserved? Does the model recommend the same items to everyone?	Relevance is traded along a frontier. NDCG@10 falls by 1.8% at the anchor weight chosen on validation and by 11.5% at the main run, most for users with short histories. Without the anchor every user receives the same list. On Yambda the main run keeps both relevance and the spread of impressions.
Q3. Can we get closer to long-term goals?	Return, likes and adoption can be predicted and rewarded, but their predictions rise by only a few percent. The largest predicted gain lies beyond anything in the log; only an online test can show whether users return.
Q4. What happens when generating several steps ahead?	A generated list hits more often after a hit and less often after a miss, and simulated responses bias the reward of later steps. Predicting each position from the real history removes the loss (+6.3% NDCG@10).
Q5. What is the trade-off between relevance and reward?	RL settings choose a point on the frontier, and inference reaches it without training. The frontier moves when training draws on the log: reward-weighted fine-tuning on the logged relative share raises the logged surplus by 21% and personalisation by 19% over the same epoch of plain training.

control [23], [24], [25]; Gao et al. [10] compare best-of-n sampling with RL along this divergence. Consumption time is confounded by item duration [26], [27], retention has been optimised directly [28], [29], and maximal marginal relevance [30] greedily optimises a list-dependent reward.

Our probes come from the post-training of language models: low-rank updates [31], which match full RL even at rank 1 [32]; the KL-anchored optimum computed at decoding time with a learned value of what follows [33], [34]; learning from logged rewards without a reward model [35], [36] and from preference pairs [37]; weight interpolation and extrapolation [38], [39], [40]; group-relative advantages [41]; and one head per future position [42].

Closest to this study, Chidambaram et al. [36] find that a learned reward model for a generative recommender predicts no better than item means. PPO and DPO against it collapse ranking metrics, while exponential reward-weighted fine-tuning on logged rewards does better. They derive the KL-anchored optimum in closed form and note that standardising rewards rescales its temperature. OneRec-V2 [43] found its reward model

Table 2: Terms used throughout.

Term	Meaning
Base recommender π_0	The pretrained GPT-2 next-item model. Every method starts from it.
Generated, ranked list	Ten items chosen one at a time, each appended to the history as if consumed; or the ten items π_0 scores highest for the real history.
Hit, miss	A recommended item that is, or is not, among the user's next 20 logged items.
Consumed share ρ	The share of a title watched or a track played (`played_ratio` in the code).
Relative share q	ρ as a quantile among the item's training-period responses. 0.5 is usual for the item, so no item earns it by being easy to finish.
Reward model	A second GPT-2 that predicts the response to an item before it is consumed. The *training* copy gives the rewards; the *held-out* copy, trained on other users, only judges.
Anchor β, KL distance	The KL penalty towards π_0 and its weight; and how far a policy ended from π_0, in nats per generated item. The distance can be compared between runs; the weight cannot.
Main run	RL on the default reward at $\beta = 0.3$. It shows a clear move along the frontier; it is not the operating point we recommend, which is chosen on validation.
Frontier	The highest predicted share reached at each level of relevance. Moving along it trades one for the other; a method that moves it gains both.
Logged share, logged surplus	Per recommended item, the response the log recorded if the item was a hit, else zero; the logged share mostly counts hits. The surplus counts $q - 1/2$ instead: it is length-neutral and positive only if users took to the items more than usual. Neither needs a model.
Personal part	The predicted share of a list for its own user minus that of the same list scored for another user. "x% of a gain is personal": the personal part's share of the gain.
Per-item share	The share of a change within one history that one effect per item explains (Equation 4): 1 if every user's items move alike.
Item offsets	RL restricted to one score offset per item, the same for every user.
Re-ranking, guided decoding	The closed-form optimum $\pi_0 \exp(r/\beta)$ computed at inference, over the candidates of π_0 at once or one position at a time.
Reward-weighted fine-tuning	Next-item training on logged items, each weighted by $\exp(r/\tau)$ of its logged response. It needs no reward model.
One head per position	A linear head per list position that predicts the item k events ahead from the real history.

open to reward hacking and moved to user feedback, scoring watch time as a percentile within the user's own history of videos of similar duration. Our reward model, in contrast, predicts responses far better than item means (AUC 0.755 against 0.656 on KION), yet RL extracts only its per-item part. We trace this before any collapse, at controlled KL distances and on two domains. We show whether the reward or the optimiser sets the per-item part, and how much of RL's gain and loss it carries. Finally, we find what moves the frontier that it defines: their reward-weighted fine-tuning with a reward that compares a response with what is typical for the item, and lists predicted from the real history.

3 Method

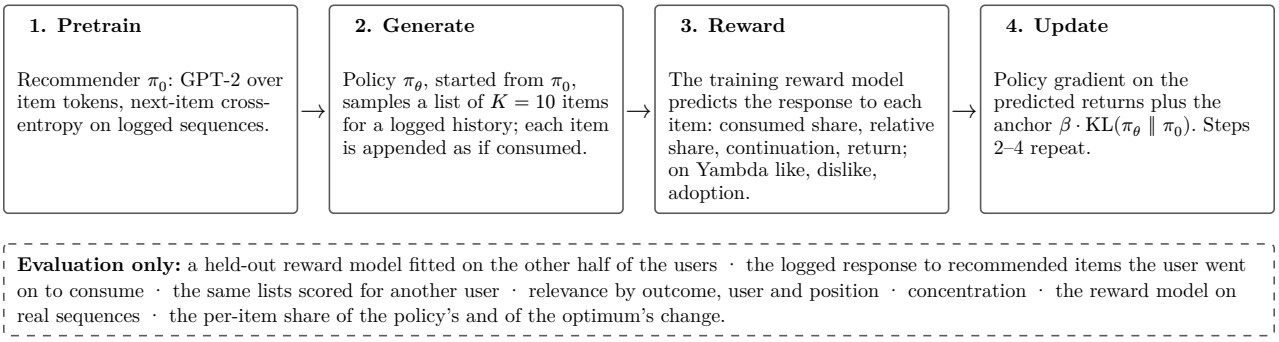


Figure 1: The pipeline. The reward model of step 3 is the only source of reward; the instruments in the dashed box never influence the policy.

Models. A log is a set of events with a user, an item, a time and the *consumed share* $\rho \in [0, 1]$ of the track or title. The recommender π_0 is a GPT-2 decoder [14] over item tokens, trained with the full-softmax next-item cross-entropy. A list of $K = 10$ is obtained by *ranking*, which scores the catalogue once, or by *generation*, which appends each chosen item to the history as if consumed [2]. RL fine-tunes the generation mode.

The *reward model* is a second GPT-2. Its token at position t carries the item i_t and the consumed share of the previous item, so it scores a candidate before its outcome exists. Its heads are trained with binary cross-entropy and predict:

- the consumed share $\hat{\rho}$, with ρ as a soft label;
- the *relative share* $\hat{q}$: the share as a mid-rank quantile among the item's training-period shares. It averages one half for every item, so an item cannot earn it by being easy to finish;
- the probabilities that the session continues, $\hat{c}$, and that the user returns, $\hat{u}$;
- on Yambda, the probabilities of a like and of a dislike, and of *adoption* $\hat{\alpha}$: a first play followed by another play in a later session within 14 days.

The log holds only items that users chose, so every head describes the response to an item *given* that the user starts it. Two copies are trained on disjoint halves of the users: the *training* model gives the rewards, the *held-out* model only judges. A third pair is trained with *hidden responses*. In half of its training windows, the responses to the last 1 to 10 events are replaced by an "unknown" value, which is the situation of a generated list.

Fine-tuning. An episode generates one list for a training history. The reward model is the environment, and it is fed its own expected response for each earlier generated item. The policy gradient uses one of three estimators: a learned value model with generalised advantage estimates [44] and four steps of PPO's clipped surrogate per batch [45]; a leave-one-out baseline over four lists per history [46]; or group-relative advantages (GRPO) over the same four lists [41]. Advantages are standardised in every case. The loss adds the exact divergence from π_0 at every position:

$$\mathcal{L}(\theta) = \mathcal{L}_{\mathrm{PG}}(\theta) + \beta \cdot \mathbb{E}_k[\mathrm{KL}(\pi_\theta(\cdot \mid s_k) \,\|\, \pi_0(\cdot \mid s_k))]. \tag{1}$$

Instead of a fixed β, a run may set a *KL budget* δ: β is then adapted after every iteration so that the policy settles at δ nats per item. Two ablations remove credit along the list: discount $\gamma = 0$, and episodes of a single item.

For a reward $r(j \mid s)$ of item j in state s, the anchored objective at one position has the closed-form maximiser [23], [36]

$$\pi^*(j \mid s) = \frac{\pi_0(j \mid s) \exp(r(j \mid s)/\beta)}{Z(s)}. \tag{2}$$

Standardising the advantages makes the policy-gradient term scale-free but leaves the KL term as it is. RL therefore pursues this optimum with an effective weight $\beta\sigma_A$, where σ_A is the scale of the advantages before standardisation [36]. Since σ_A depends on the estimator and drifts during training, the weight β cannot be compared between runs. The KL distance a run ends at can, and we report it throughout.

Rewards. All rewards are instances of

$$r_k = w_\rho\hat{\rho}_k + w_q\hat{q}_k + w_u\hat{u}_k + w_L\left(\hat{l}_k - \hat{d}_k\right) + w_A\hat{\alpha}_k + w_R\mathbb{1}[k=K]\hat{u}_K + w_F\frac{\mathbb{1}[a_k \in F]}{\log_2(k+1)}. \tag{3}$$

The default is $w_\rho = 1$ alone. The *return after the list* (w_R) is paid at the last item only, so it belongs to the list as a whole. The *logged-relevance* term of [8] (w_F) uses the user's real continuation F of a training history and sums to DCG@K. A *personal* reward subtracts what the model predicts for the same list shown to another user, and a *diversity* reward adds the mean distance between the items of the list. The return, like and adoption terms are studied in Section 5.3.

Alternatives to on-policy RL (Section 5.3, Section 5.5). *Liked-only fine-tuning* continues next-item training on targets with $\rho \geq 0.5$. *Off-policy REINFORCE* [9] weights logged items by the logged reward that followed them, with importance weights capped at $\kappa = 0.5$. *Re-ranking* keeps the K items, of the 100 best candidates of π_0, with the highest $r(j \mid h) + w\log\pi_0(j \mid h)$. This orders items exactly as π^* of Equation 2 with $\beta = w$, so re-ranking computes the top of the anchored optimum for the real history, without training. *Guided decoding* applies Equation 2 one position at a time. Among the 50 most likely next items of π_0, it takes the one with the highest $\log\pi_0 + Q/\beta$, given the items already chosen; we call β here the *strength* of guidance. Q is the item's predicted reward plus γ times a value U of what follows it, and U is regressed on the discounted rewards of lists that π_0 samples. This is controlled decoding [33], the first step of Q♯ [34]. With $\gamma = 0$, each item is judged by its own reward alone. *Weight interpolation* takes $\theta_0 + a(\theta_{\mathrm{RL}} - \theta_0)$ for a from 0.25 to 2 [38], [39], [40].

What RL learns (Section 5.1). *Item offsets* restrict RL to one score offset per item, the same for every user. *Rank-1 LoRA* [31] updates only the transformer's projections. Both are held to the KL budgets of full RL, with learning rates of $3 \cdot 10^{-3}$ and 10^{-4} against full RL's 10^{-5}. *Policy surgery* averages the main run's change of log-probabilities per item over validation histories. Adding this average to π_0 gives "base + RL's item offsets"; subtracting it from the fine-tuned policy gives "RL − its item offsets".

Learning from the log (Section 5.5). *Reward-weighted fine-tuning* continues next-item training with each logged target weighted by $\exp(r/\tau)$, normalised over the batch, where r is the logged relative or consumed share. It fits Equation 2 to the log [35], [36]. *Within-user DPO* [37] learns from pairs of items that one user consumed within 20 events after the same history: one with a relative share of at least 0.6 and one of at most 0.4. Neither item is therefore preferred for having been started or for being short. Both methods run one epoch at a learning rate of 10^{-4}.

One head per position (Section 5.4) [42] predicts a list without reading its own items. On the frozen π_0, a linear map per offset $k = 2, ..., K$, initialised as the identity, turns the state after the real history into one from which π_0's output layer scores the item k events ahead. Position k takes head k's best allowed item.

Instruments. Let F be the next 20 items the user really consumed; a recommended item in F is a *hit*.

- *Held-out reward model.* A gain it does not confirm is exploitation of the training model.
- *Logged share and logged surplus.* Neither needs a model. The logged share $\frac{1}{K}\sum_k \mathbb{1}[a_k \in F]\rho(a_k)$ counts hits as much as responses. The logged surplus $\frac{1}{K}\sum_k \mathbb{1}[a_k \in F]\big(q(a_k) - \frac{1}{2}\big)$ is positive only if users took to the recommended items more than is usual for those items, so an item's length does not raise it.
- *Personal part.* The same lists scored for another user give the part of the predicted share that depends on who receives the list.
- *Relevance* is NDCG, Recall and MAP at $K = 1, 5, 10$ against F. NDCG@10 or Recall@10 is also broken down by outcome (liked, $\rho \geq 0.5$, or skipped; above or below the item's usual share), by the length of the history, and by position after a hit or a miss.
- *Concentration* is catalogue coverage, the Gini coefficient of impressions, the share of impressions of the 1% most shown items, and the overlap of two users' lists.
- *Per-item share.* For a quantity x_{hj} of candidate j after history h, let $\tilde{x}_{hj}$ be x_{hj} minus its mean within the history, with weights $w_{hj} = \pi_0(j \mid h)$ over the 100 best candidates of π_0. The per-item share is the part of the within-history variation that one effect e_j per item explains:

$$S = 1 - \frac{\sum_{h,j} w_{hj}\big(\tilde{x}_{hj} - e_j\big)^2}{\sum_{h,j} w_{hj}\tilde{x}_{hj}^2}, \quad e_j = \frac{\sum_h w_{hj}\tilde{x}_{hj}}{\sum_h w_{hj}}. \tag{4}$$

Table 3: Data, split, models and fine-tuning.

	KION (main dataset)	**Yambda-50M (replication)**
Log	Online cinema [47]: one row per user and title, dated by its last view, with the share watched; an item table names every title and its type	Music service [48]: one row per listen, timed to the second, with the share played; likes and dislikes
Kept	Users with at least 5 and titles with at least 10 training events	Users with at least 50 training events; the 50k most played tracks
Size	256k users, 8k titles, 3.5M training events; a median of 10 per user	8.7k users, 50k tracks, 35M training events; a median of 2,638 per user
Split	The last 7 days are the test period, the 7 days before them the validation period	The same
Test requests	59,997, one per user at the start of the test week; the history is the last 38 events, the targets the next 20	7,593; the last 118 events and the next 20
Session, return	The next event on the same calendar day; a new day within a week	The next event within 30 minutes; a new session within a day
Models	GPT-2, 4 layers of width 256, windows of 48 events	The same, windows of 128 events
Fine-tuning	512 lists of 10 per iteration, $\gamma = 0.99$, learning rate 10^{-5}; the main run has $\beta = 0.3$ and 2,000 iterations, every other run 1,000	The same
History items	Never recommended again: one row per user and title	Allowed: repeats are normal
Chosen on validation	$\beta = 1$	$\beta = 0.05$

$S = 1$ if every history orders the items alike, and $S = 0$ if an item's value depends on the history alone. For $x = \log \pi_\theta - \log \pi_0$, S measures how much of a policy's change is one preference per item applied to every user. For the reward model's predictions, S measures the same for the optimum: by Equation 2 the optimum changes $\log \pi_0$ by r/β plus a constant per history, at any β.

- *Simulation on real sequences* runs the reward model on the user's real next five items, with the earlier responses as logged or as a simulation has to assume them.

4 Experimental Setup

Data. Table 3 lists the data, the split and the models. On Yambda 0.77% of events are liked and 0.16% disliked, and 24% of first plays are adopted. The consumed share is bimodal, and on KION it depends on length. Films are watched to 0.5 on average. The longest tenth of the catalogue, mostly series, is watched to 0.22, since the share is taken of the whole series.

Training and evaluation. Fine-tuning cuts its histories at a random moment of the training period of a random user with at least 8 training events. Lists of ten are generated greedily unless ranked lists are named, and every baseline reads the recommender's window of history. The KL distance of a run is the exact divergence over the catalogue on its own sampled training lists, averaged over the last 50 iterations. Operating points are selected on validation requests only: the highest held-out predicted share within 5% of the base recommender's NDCG@10. The main run's $\beta = 0.3$ is not such an operating point. It was carried over from earlier iterations of the study, and we keep it because it moves clearly along the frontier, which makes its changes easy to analyse.

No leakage. Every trained model reads windows of the training period only: the recommender, the reward models, the baselines and every kind of fine-tuning. RL samples its histories, and the logged continuation of each, inside the training period. The catalogue and every item statistic, including the quantiles of the relative share, come from the training period. Labels that look ahead, such as continuation, return, likes and adoption, are computed for training events from events before the validation period; an outcome whose waiting time is not observed is left out. Early stopping and operating points use the validation period. The history of a test request includes the validation week, on which no model trained. On KION a title watched across the boundary appears only as a test target, and test events on items outside the training catalogue are dropped for every method alike. The repository's README lists each check next to the code that implements it.

Sessions and seeds. The experiments ran as several sessions per dataset, and every table and figure names the session it comes from (Appendix A). Seeds 0, 1 and 2 repeat every central comparison. "$x \pm s$" is the mean and standard deviation over the three seeds, with s in the unit of x (percentage points when x is a percentage); each quantity is computed within its seed. Other numbers are from seed 0. Every conclusion below holds in each seed.

Table 4: The main run ($\beta = 0.3$) against the base recommender, final session, seed 0. Changes are relative, in percent, with a 95% paired-bootstrap interval over users where the quantity is defined per user.

	KION			Yambda		
	before	**after**	**change, %**	**before**	**after**	**change, %**
Relevance against the user's logged future						
NDCG@10, all next items	0.1173	0.1047	−10.7 [−11.6, −9.8]	0.1172	0.1183	+0.9 [−0.7, +2.5]
NDCG@10, liked items	0.1214	0.1229	+1.3 [+0.1, +2.4]	0.1005	0.1029	+2.4 [+0.6, +4.3]
NDCG@10, items consumed above usual	0.1261	0.1142	−9.4 [−10.6, −8.2]	0.0998	0.1023	+2.5 [+0.7, +4.3]
Recall@10, skipped items	0.1382	0.1130	−18.2 [−19.6, −16.8]	0.0724	0.0735	+1.5 [−1.7, +4.6]
Predicted consumed share of the list						
Training reward model	0.417	0.502	+20.3 [+20.1, +20.4]	0.594	0.608	+2.2 [+2.1, +2.4]
Held-out reward model	0.455	0.553	+21.5 [+21.4, +21.7]	0.599	0.609	+1.7 [+1.5, +1.9]
Held-out, list shown to another user	0.442	0.539	+22.0 [+21.9, +22.2]	0.498	0.508	+2.0 [+1.7, +2.3]
Held-out, relative share	0.529	0.524	−1.0 [−1.1, −1.0]	0.469	0.468	−0.3 [−0.4, −0.1]
Logged response to the recommended items the user went on to consume						
Such items per list	0.359	0.329	−8.3	0.943	0.961	+1.9
Their logged share	0.481	0.528	+9.8	0.710	0.713	+0.5
Logged share per recommended item	0.0173	0.0174	+0.6 [−0.6, +1.7]	0.0669	0.0686	+2.5 [+0.3, +4.6]
Logged surplus per recommended item	0.0012	0.0005	−61.9 [−69.4, −54.4]	0.0033	0.0032	−3.0 [−15.2, +9.1]
Lists						
Catalogue coverage	0.495	0.449	−9.2	0.519	0.506	−2.6
Overlap between two users' lists	0.2092	0.2509	+20.0	0.0014	0.0020	+42.9
Mean duration of an item, minutes	339.1	205.7	−39.3	3.4	3.3	−3.0

5 Results

Base models. On KION the recommender's ranked lists are level with GRU4Rec, 13% above SASRec and 5% below BERT4Rec (Table 16). On Yambda it is the best trained model. Replaying the user's own history from the same 118 events does better still (NDCG@10 0.187 against 0.117), mostly through repeated tracks. On Yambda we therefore report relevance as a change against this recommender, not against the best list possible. The reward model predicts whether a held-out event is consumed at least half with AUC 0.755 on KION and 0.868 on Yambda; the item's mean share reaches 0.656 and 0.607. The like and dislike heads reach 0.74 and 0.73. The two copies are calibrated on their own users and differ in level on the same lists (0.417 and 0.455 for the base recommender on KION), so we compare their relative changes.

5.1 Q1. How does the model's behaviour change after RL fine-tuning?

Is the reward maximised? As far as the reward models can tell, yes. On KION the main run raises the consumed share predicted by the training reward model by 20.3%, and by the held-out one by 21.5% (seed 0; $23.9\% \pm 2.1$ over three seeds; Table 4). The two models rise together from the first evaluation. Most of the change comes early: 85% of the gain and 90% of the loss of NDCG@10 arrive within the first 250 of 2,000 iterations (Figure 2). After that the policy keeps moving away from π_0, from 0.06 to 0.09 nats per item, for little more reward.

Do users respond better? The log says no. The logged surplus, users' response to recommended titles beyond what is usual for each title, falls by $61\% \pm 2$, from 0.0012 to 0.0005 per recommended title. At the validation-selected $\beta = 1$ it falls by 15%. The logged share per recommended item, which mostly counts hits, rises by $1.1\% \pm 0.4$. Users reached fewer of the recommended titles (hits $-8.8\% \pm 0.9$) but finished those more often ($+10.8\% \pm 1.6$), because they are titles that everybody finishes.

How the recommendations differ. The lists shift towards films. Films take 66% of the impressions of the base recommender's generated lists and 80% of the main run's; at $\beta = 0.1$ they take 89%, and without an anchor all of them. The titles that gained most are films of 1.4 to 2.2 hours, usually watched to 0.54–0.69. Four of the five that lost most are series of 9.5 to 22 hours, usually watched to 0.08–0.29 (Table 12). Popularity is not what the policy follows: one of the largest gains is a film that only 953 users watched in the training period. Figure 3 shows the shift across the catalogue. The four tenths of titles with the lowest usual share fall from 34% to 13% of the impressions, and the three highest rise from 29% to 46%. The longest tenth, mostly series, falls from 26% to 14%. On Yambda impressions move the same way, by a few points.

One re-weighting of the catalogue. Three tests show that the policy's change on KION is mostly one preference per item, the same for every user (Table 5).

- *Restricting RL to item offsets loses little.* Held to the same KL budget as full RL, RL that may change only one score offset per item reaches $93\% \pm 3$ of full RL's gain in predicted share at a budget of 0.1 nats, and $89\% \pm 4$ at 0.5. Its NDCG@10 is within 0.002 of full RL's.

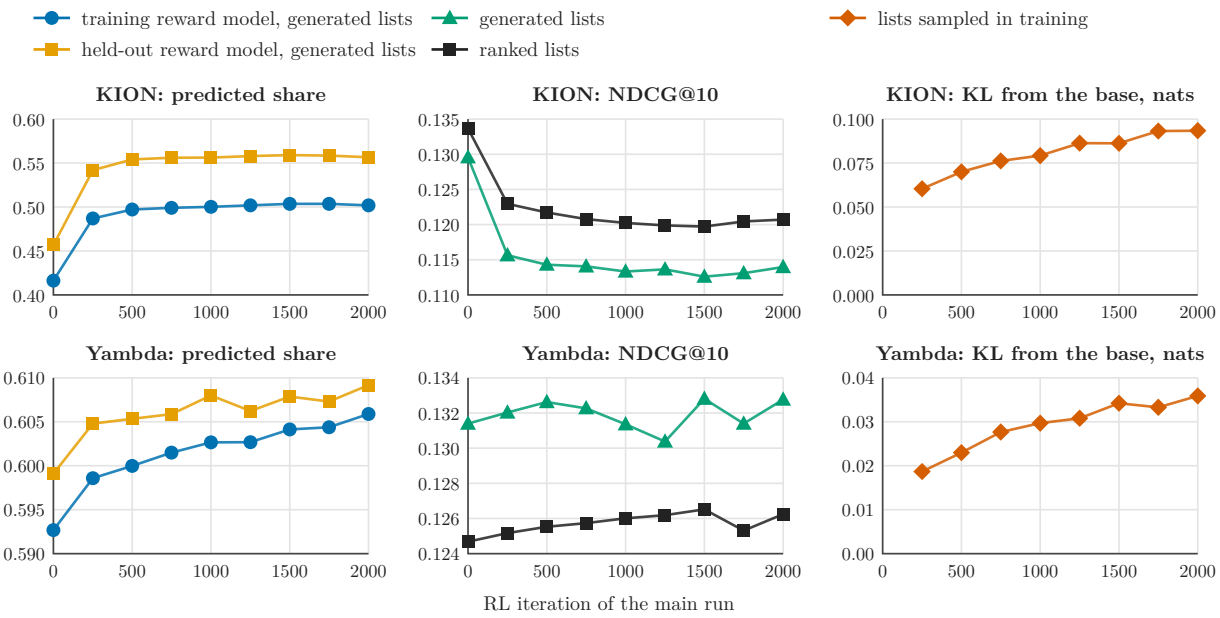


Figure 2: The main run over its iterations (final sessions): the predicted share of the generated lists for validation requests as each reward model predicts it; NDCG@10 of the generated and ranked lists for the same requests; and the KL from the base recommender, in nats per generated item, of the lists sampled in training, recorded from the first update of the policy.

- *The gain does not depend on the user.* The same lists scored for another user earn the same gain: only $0.2\% \pm 0.5$ of it is personal. Re-ranking by each item's mean share in the training log, with no reward model, reaches the main run's point (predicted share 0.549 at NDCG@10 0.109, against 0.553 at 0.105).
- *The per-item part carries the whole effect.* Adding the main run's average change per item to π_0 reproduces $88\% \pm 2$ of its gain and all of its loss of relevance (NDCG@10 $-11.0\% \pm 1.8$, against RL's own $-11.6\% \pm 1.5$ in the same sessions). Subtracting it from the fine-tuned policy leaves $21\% \pm 2$ of the gain at the base relevance ($-0.1\% \pm 0.4$).

The two parts pull personalisation in opposite directions. The per-item part lowers the personal part of the predicted share by $15.7\% \pm 3.1$, the rest raises it by $20.5\% \pm 2.5$, and in RL they nearly cancel ($+2.2\% \pm 2.9$). What the per-item part ranks is clear from its correlations: an item's mean shift correlates 0.90 with its usual

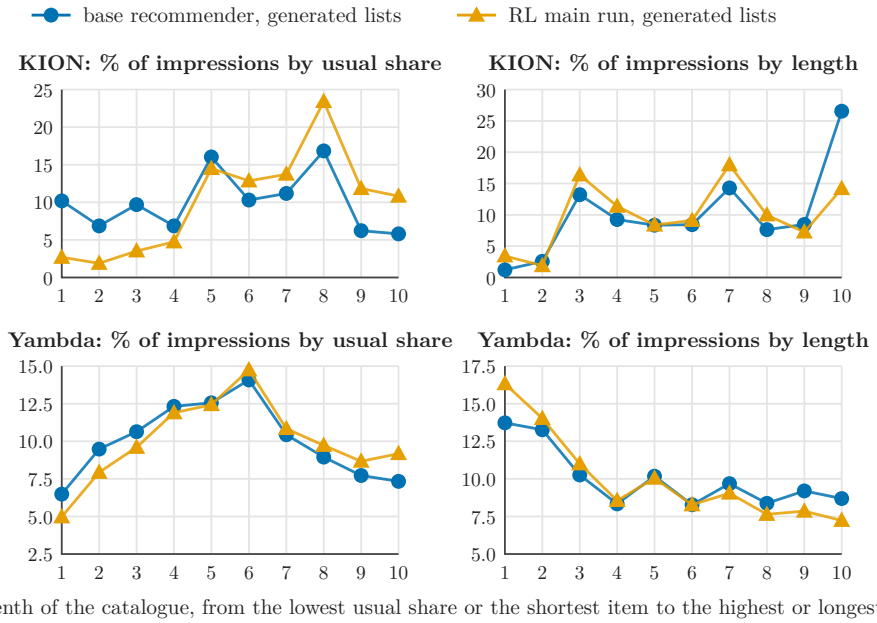


Figure 3: Where the impressions went (final sessions): the share of all impressions in the test users' lists that each tenth of the catalogue received from the base recommender and from the main run. The catalogue is cut into tenths with as many items each, by the item's usual consumed share in the training period and by its length.

Table 5: What RL learns (frontier sessions). Restricted runs and full RL hold the same KL budget, except that item offsets end short of 0.5: at a KL of 0.41 on KION (full RL 0.53), and on Yambda at 0.25 with the anchor weight at zero. The parts of the main run are compared with the base recommender. The per-item share (Equation 4) is measured on the base recommender's 100 best candidates for every test user; that of the reward model's predictions is the optimum's at any β (Equation 2).

	KION, 3 seeds	**Yambda, 1 seed**
Fine-tuning held to a KL budget: % of full RL's gain in predicted share		
One score offset per item, budget 0.1	93 ± 3	–
One score offset per item, budget 0.5	89 ± 4	34
Rank-1 LoRA, budget 0.1	87 ± 5	–
Rank-1 LoRA, budget 0.5	93 ± 6	95
The main run's change split into one offset per item and the rest		
Base + RL's item offsets: % of RL's gain	88 ± 2	52
Base + RL's item offsets: NDCG@10, change in %	-11.0 ± 1.8	+0.4
Base + RL's item offsets: personal part, change in %	-15.7 ± 3.1	+0.0
RL − its item offsets: % of RL's gain	21 ± 2	50
RL − its item offsets: NDCG@10, change in %	-0.1 ± 0.4	+0.3
RL − its item offsets: personal part, change in %	$+20.5 \pm 2.5$	+1.1
RL: NDCG@10, change in %	-11.6 ± 1.5	+0.6
RL: personal part, change in %	$+2.2 \pm 2.9$	+1.4
Share of within-history variation that one effect per item explains		
Reward model's predictions (the optimum's change)	0.79 ± 0.01	0.42
RL, main run	0.77 ± 0.01	0.50
RL, KL budget 0.5	0.73 ± 0.02	0.49
Rank-1 LoRA, budget 0.5	0.72 ± 0.01	0.52
Item offsets, budget 0.5 (their ceiling)	0.89 ± 0.00	0.85

consumed share, 0.12 with its log popularity and −0.36 with its duration (review session). The policy learned a ranking of titles by how often they are finished, and it applies this ranking to every user.

The reward or the optimiser? Equation 2 answers this before any RL. The per-item share of the reward model's predictions is that of the optimum's change, at any β. On KION it is 0.79 ± 0.01. Every run of the anchor sweep, the KL budgets and LoRA stays below it, in every seed (main run 0.77 ± 0.01). On KION the reward thus sets the re-weighting, and the optimiser follows it.

The reason lies in what the reward measures. $\hat{\rho}$ is the share a user consumes *given* that they start the item, so it cannot penalise an item the user would never start. And on KION the share of a series is taken over all its episodes. An item that everybody finishes therefore earns a high reward, whoever receives it.

On Yambda the reward is mostly personal: its per-item share is 0.42. Yet every RL run is more per-item than its reward (0.46 to 0.55). A policy gradient averages a noisy signal over histories, and the direction that is the same for all of them accumulates first. Item offsets alone fall short on Yambda: even with the anchor weight driven to zero they stop at a KL of 0.25. At a budget of 0.5 they reach 34% of full RL's gain, less than full RL reaches at a KL of 0.11. The reward model also knows more about the user than RL uses: $55\% \pm 9$ of re-ranking's gain is personal, against $16\% \pm 16$ of RL's. Rank-1 LoRA stays on full RL's frontier on both datasets, with 87% to 95% of its gain at a higher NDCG@10, as reported for language models [32].

Answer to Q1. The reward is maximised as far as the reward models can tell. But what RL learns is one re-weighting of the catalogue, the same for every user, and this re-weighting carries RL's whole loss of relevance. Measured against what is usual for each title, users took to the recommendations less. On KION the reward sets the re-weighting, since 79% of its optimum's change is per item. On Yambda, where the reward is mostly personal, the optimiser amplifies the per-item part.

5.2 Q2. Is relevance preserved? Does the model recommend the same items to everyone?

A frontier, not a verdict. How much relevance is kept depends on the operating point (Figure 4). On KION the weight selected on validation requests is $\beta = 1$. There NDCG@10 falls by 1.8%, while the predicted share rises by 8.3% and the logged share by 3.3% (three seeds). At the main run, $\beta = 0.3$, NDCG@10 falls by $11.5\% \pm 1.3$ for a 24% higher predicted share. On Yambda the validation rule selects $\beta = 0.05$ (NDCG@10 −4.9%, predicted share +5.0%). This is one grid step from $\beta = 0.03$, which collapses the policy in every seed. The Yambda main run moves little.

Table 6: Ranking metrics of the main run against the base recommender (generated lists, final session). At $K = 1$ NDCG and MAP coincide.

	KION			Yambda		
	before	**after**	**change, %**	**before**	**after**	**change, %**
NDCG						
NDCG@1	0.0885	0.0814	−7.9	0.1667	0.1655	−0.7
NDCG@5	0.1044	0.0920	−11.9	0.1336	0.1342	+0.4
NDCG@10	0.1173	0.1047	−10.7	0.1172	0.1183	+0.9
Recall						
Recall@1	0.0439	0.0389	−11.4	0.0159	0.0157	−1.3
Recall@5	0.1220	0.1050	−14.0	0.0507	0.0508	+0.2
Recall@10	0.1620	0.1453	−10.3	0.0747	0.0756	+1.2
MAP						
MAP@1	0.0885	0.0814	−7.9	0.1667	0.1655	−0.7
MAP@5	0.0781	0.0682	−12.6	0.0997	0.0996	−0.1
MAP@10	0.0819	0.0719	−12.2	0.0753	0.0754	+0.1

Relevance by Recall, NDCG and MAP. At the main run on KION every metric falls at every cut-off, by 8 to 14% (Table 6). The loss already appears at $K = 1$ (−7.9%). The first item is chosen from the real history alone, so this part of the loss is the policy's own preference; from the second position on, the generated context adds to it (Section 5.4). The loss is uneven across outcomes. NDCG@10 on liked titles is kept (+1.3%), while Recall@10 on skipped titles falls by 18.2% and NDCG@10 on titles watched above their usual share by 9.4%. It is also uneven across users: those with the least history lose most, −13.5% of NDCG@10 for 5 to 8 earlier events against −7.4% for 25 or more (Table 13). On Yambda every metric stays within 1.3% of the base recommender, and no group of users loses.

The same items for everyone? Not at the main run, but closer to it (Table 14). On KION the base recommender's generated lists already give 72% of the impressions to the 1% most shown titles (62% for ranked lists). The main run raises this to 76%, and the Gini coefficient of impressions from 0.963 to 0.969. A weaker anchor makes the lists converge: 92% at $\beta = 0.1$. Without an anchor nearly every user receives the same list (overlap between two users' lists 0.98 on KION, 1.00 on Yambda), and the held-out model still scores it 51% above the base recommender. On Yambda the share of the top 1% moves only from 18% to 20%. What makes the lists alike is the per-item part of the policy (Section 5.1).

What keeps relevance. Only the anchor asks for relevance, but its weight does not say how much relevance it keeps; the KL distance does. Two estimators at the same $\beta = 0.3$ on KION end far apart: the leave-one-out estimator ends 6.7 times further from π_0 than the value model (KL 0.543 ± 0.010 against 0.081 ± 0.004). Held to the same KL budget of 0.1, the two end at the same relevance (NDCG@10 0.1037 ± 0.0021 and 0.1013 ± 0.0026), with weights 2.9 times apart. The effective weight $\beta\sigma_A$ (Section 3) explains most of this. Before standardisation, the value model's advantages are 2.2 times larger at that budget (σ_A 0.95 against 0.43), and 1.7 times larger at a budget of 0.5, where the weights differ 2.7 times. On a logarithmic scale this accounts for 55% to 75% of the gap between the weights. On Yambda $\beta = 0.1$ keeps relevance, and $\beta = 0.03$ collapses the policy in every seed (NDCG@10 0.006 ± 0.004; Table 15).

> **Answer to Q2.** Relevance is traded along a frontier. On KION, NDCG@10 falls by 1.8% at the validation-selected weight and by 11.5% at the main run, at every cut-off and most for users with short histories, while impressions concentrate further. The per-item part of the policy causes both. On Yambda the main run keeps both relevance and the spread of impressions. The anchor is what keeps relevance, and the KL distance a run ends at tells how much it keeps far better than the anchor weight does. Without the anchor every user receives the same list.

5.3 Q3. Can we get closer to long-term goals?

Which signals. Four logged signals describe what follows an item: the session continues, the user returns, the user likes or dislikes the item, and the user adopts it. The last two exist on Yambda only. A reward can steer a policy only if it differs between the items a user could be shown. The predicted return barely does: it varies between one user's candidates four times (KION) and nine times (Yambda) less than between users. The probability of a like varies by only 0.003 between a user's candidates. Adoption is the exception. It varies between candidates half as much as between users (0.077 against 0.165), and it is the only signal that leans towards the items users really consumed next (AUC 0.56).

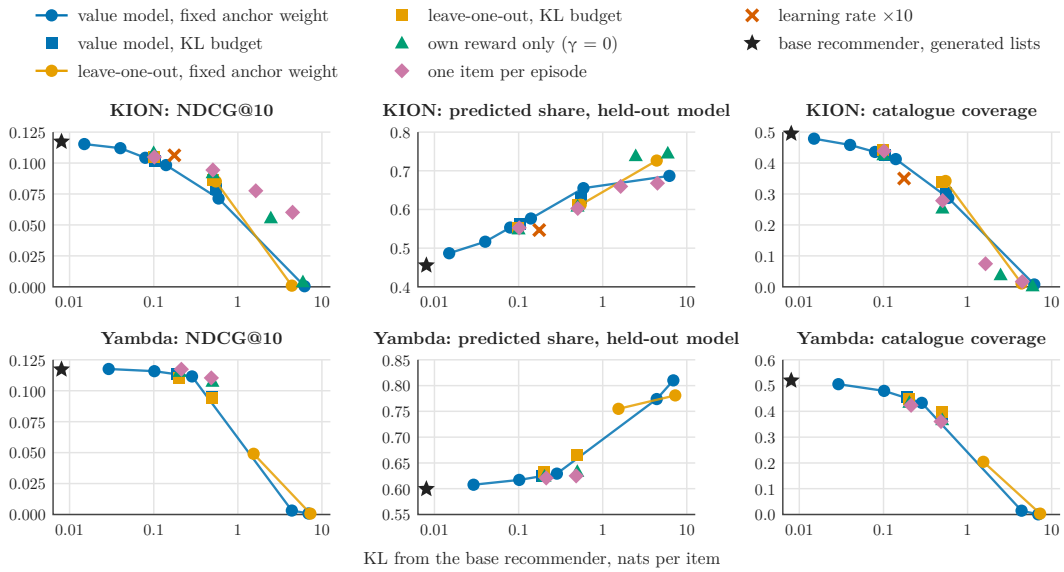


Figure 4: Every fine-tuning run of the default reward against the KL distance it ended at from the base recommender (logarithmic axis; the base recommender at the left edge; final session).

Rewards for return and continued interaction. We rewarded each signal per item, and the return also once per list, after its last item. Rewarding the return after each item raises it by 3 to 7% on KION, at a cost of 8 to 23% of NDCG@10, and does not move it on Yambda (Table 19).

The *return after the list* moves the KION policy further than any per-item reward (KL 0.51 at $\beta = 0.3$). The predicted return rises from 0.73 to 0.86 (+18%), and the held-out model agrees. But NDCG@10 falls by 69%, the logged share by 64%, and the mean duration of a recommended title from 5.7 to 2.2 hours (Table 7). Nothing in the log supports a return rate of 0.86. Neither title length nor KION's format, one row per user and title dated by its last view, produces it: in every decile of the duration of an event's own title or of the next title, the logged return rate lies between 0.68 and 0.77. Both reward models extrapolate beyond the log in the same way, so the held-out model cannot catch it. This is the one reward for which credit along the list matters: with $\gamma = 0$ the policy hardly moves (KL 0.006). On Yambda the return after the list does not move.

Likes and adoption (Yambda). The like reward at $\beta = 0.3$ raises the predicted probability of a like from 1.08% to 1.17%, for 2% of NDCG@10. At $\beta = 0.1$ likes rise by 49%, but dislikes rise by 53%: the policy finds tracks that provoke reactions, not tracks that are liked more. Adoption is the one long-term reward that moved without a loss of relevance. At $\beta = 0.3$ its predicted probability rises from 0.233 to 0.277 (+19%), NDCG@10 by 1.5% and the logged share by 2.5%, while coverage falls from 0.52 to 0.41.

Steering at inference. Guided decoding steers the base recommender towards these rewards without fine-tuning it (Table 7). On Yambda, guidance by predicted adoption raises adoption by 6.5% and NDCG@10 by 3.7% at strength 0.3. At strength 0.1 it raises adoption by 20% for 0.8% of NDCG@10; the RL run gained 19% of adoption and 1.5% of NDCG@10. Judging each track also by a learned value of what follows it adds 3 and 6 points of adoption. On KION the return after the list is paid only at the end, so guidance that judges each title by its own reward sees it only at the last position. Look-ahead nearly doubles the gain: at strength 0.1 the predicted return rises from 0.729 to 0.742 without it and to 0.753 with it, for 2.3% of NDCG@10. RL reached 0.86 but lost 69% of NDCG@10. The guided lists also keep the length of their titles.

Offline or online. Offline we can check whether a signal is predictable, whether it varies between the items a policy chooses from, whether a held-out model agrees, and what it costs in relevance. Whether users return, continue or adopt more under a new policy is counterfactual, for three reasons. The log shows these outcomes only under the policy that produced it. Adoption needs 14 days after a first play, which a test week does not have. And a prediction can leave the log's range, as the return after the list did. This needs the online experiment of Section 6.1.

> **Answer to Q3.** Rewards for return, likes and adoption can be learned and optimised, but at a moderate KL distance their predictions rise by only a few percent, and the largest gain lies outside the log. Adoption is the most promising signal: steering the base recommender by it at inference, with a learned look-ahead, raises adoption and relevance together. Only an online test can show whether users return or adopt more.

Table 7: Long-term rewards (review session), and guided decoding by them, which leaves the recommender as it is (frontier session; "β": the strength of guidance). "Δ": change of NDCG@10 in percent; "Return": the held-out model's predicted return after the last item; P(like), P(adopt): its predicted probabilities (Yambda only); "Cover.": catalogue coverage; "Hours": mean duration of a recommended KION title.

	KL	NDCG@10	Δ, %	Return	P(like)	P(adopt)	Logged	Cover.	Hours
KION									
Base recommender	0.000	0.1173	+0.0	0.728	–	–	0.0173	0.495	5.7
Return after list, β = 0.3	0.507	0.0361	−69.2	0.858	–	–	0.0062	0.539	2.2
Return after list, β = 0.1	3.423	0.0012	−99.0	0.888	–	–	0.0002	0.059	1.7
same, discount 0, β = 0.3	0.006	0.1144	−2.5	0.733	–	–	0.0171	0.510	5.3
Yambda									
Base recommender	0.000	0.1172	+0.0	0.892	0.0108	0.233	0.0669	0.519	–
Like − dislike, β = 0.3	0.046	0.1148	−2.0	0.891	0.0117	0.234	0.0654	0.505	–
Like − dislike, β = 0.1	0.829	0.0841	−28.2	0.878	0.0161	0.207	0.0475	0.423	–
Adoption, β = 0.3	0.141	0.1189	+1.5	0.893	0.0121	0.277	0.0686	0.409	–
Adoption, β = 0.1	1.204	0.0737	−37.1	0.888	0.0164	0.342	0.0407	0.129	–
Return after list, β = 0.3	0.021	0.1170	−0.2	0.892	0.0107	0.233	0.0677	0.521	–
KION, guided decoding by the return after the list (frontier session)									
Base recommender	0.000	0.1173	+0.0	0.729	–	–	0.0173	0.495	5.7
Guided, own reward, β = 0.3	–	0.1173	+0.0	0.734	–	–	0.0173	0.494	5.6
Guided, own reward, β = 0.1	–	0.1172	−0.1	0.742	–	–	0.0173	0.496	5.6
Guided, look-ahead, β = 0.3	–	0.1162	−0.9	0.738	–	–	0.0172	0.506	5.6
Guided, look-ahead, β = 0.1	–	0.1146	−2.3	0.753	–	–	0.0171	0.518	5.4
Yambda, guided decoding by adoption (frontier session)									
Base recommender	0.000	0.1172	+0.0	0.890	–	0.237	0.0669	0.519	–
Guided, own reward, β = 0.3	–	0.1216	+3.7	0.889	–	0.252	0.0705	0.491	–
Guided, own reward, β = 0.1	–	0.1163	−0.8	0.889	–	0.283	0.0678	0.410	–
Guided, look-ahead, β = 0.3	–	0.1221	+4.2	0.890	–	0.260	0.0706	0.475	–
Guided, look-ahead, β = 0.1	–	0.1137	−3.0	0.886	–	0.297	0.0663	0.360	–

5.4 Q4. What happens when generating several steps ahead?

A generated list appends each chosen item to the history as if the user had consumed it. It thus plans several steps ahead under an assumption that is false for most items: at most 17% of the items at any position are hits.

Effect on the following recommendations. After a miss, a generated list continues from a context that did not happen, and it hits less often than a ranked list: 0.0273 against 0.0312 at position 5 on KION, 0.040 against 0.063 on Yambda (Figure 5). After a hit it hits more often: 0.112 against 0.093 at position 2 on KION. On Yambda, where listening continues within albums and artists, the effect is larger: 0.54 against 0.25 at position 5. On KION misses dominate, and generation costs 4% of NDCG@10 against ranking. On Yambda generation is ahead at positions 2 to 7.

A list predicted position by position from the real history removes the loss. After a miss, one head per position hits at least as often as a ranked list (0.033 at position 5 on KION, 0.073 on Yambda); after a hit, as often

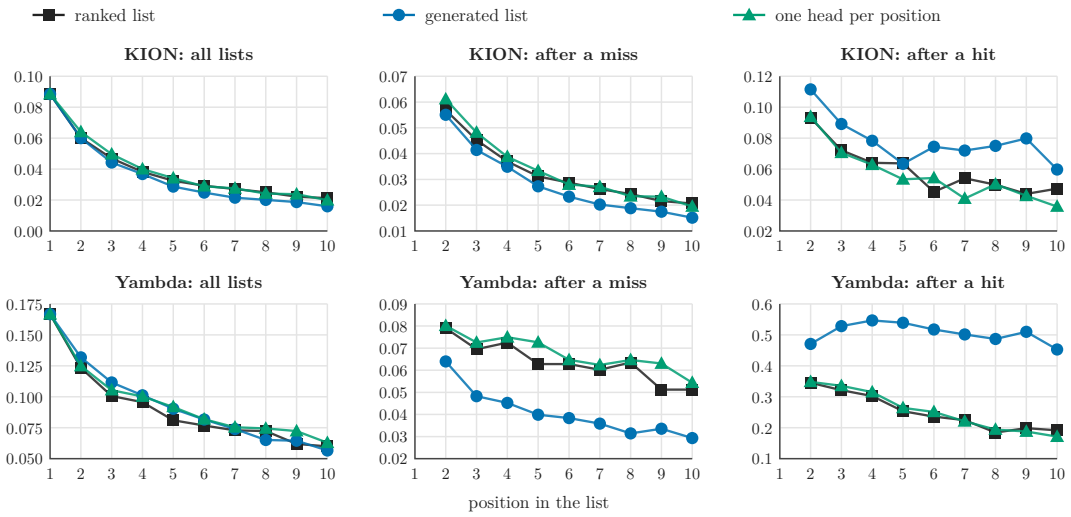


Figure 5: Hit rate by position of the list, for all lists, after a miss and after a hit at the position before: the base recommender's ranked and generated lists, and the lists of one head per position (frontier sessions).

as a ranked list. On KION it is $6.3\% \pm 0.4$ more relevant than generation and $2.5\% \pm 0.1$ more than ranking, with a logged surplus $25\% \pm 1$ above generation's. On Yambda, where hits after hits matter more, it is level with generation (+0.4%) and 3.4% above ranking, with less coverage.

Steering shows the same pattern. Guided decoding decides each position after the items already chosen, as generation does. On KION it falls slightly behind re-ranking (predicted share at the main run's NDCG@10: 0.556 ± 0.024 against 0.564 ± 0.025; RL 0.548 ± 0.023). On Yambda, where generation beats ranking, it beats re-ranking (Figure 6).

Effect on the reward estimate. The reward at later positions depends on what the reward model assumes about the responses to earlier items. We test these assumptions on the users' real next five items (Figure 7 in the appendix). With the logged responses fed back, the AUC of the predicted share rises along the list (0.757 to 0.782 on KION, 0.640 to 0.819 on Yambda). Under any simulated response it stays near its first-step level. Simulated responses also bias the level of the prediction on Yambda. Expected responses underestimate the next share by 0.06 to 0.09, sampled ones overestimate it slightly (at most +0.035), and assuming that earlier items were finished overestimates it by 0.06 to 0.18. The reward model trained with hidden responses and fed "unknown" is nearly unbiased (+0.001 to +0.013). It also reads observed responses as well as the standard model (AUC 0.755 against 0.758 on KION). Policies fine-tuned against it, or with sampled responses, land on the same frontier: the assumption sets the level of a multi-step predicted reward, not which policy wins.

Does planning help? Not for the default reward. At an equal KL budget, crediting each item only with its own reward is at least as relevant as crediting it with the rest of the list. On KION, NDCG@10 is 0.1070 ± 0.0027 against 0.1013 ± 0.0026 (one-item episodes 0.1043 ± 0.0024), at predicted shares 1.4 to 1.6 points lower. On Yambda it is 0.1076 ± 0.0014 against 0.0960 ± 0.0008. The myopic policies lie on the same frontier of relevance against reward. Only the return after the list, paid at the end, needed full credit (Section 5.3). Look-ahead at inference agrees. On KION a learned value of what follows lowers the predicted share that guided decoding reaches at the main run's NDCG@10 (0.548 against 0.567, seed 0). It helps only rewards that depend on what follows an item: the return after the list and adoption.

> **Answer to Q4.** A generated list hits more often after a hit and less often after a miss. Simulated responses bias the reward of later steps, unless the reward model was trained to read unknown ones. One head per position predicts the list from the real history and removes the loss after a miss: it gives the most relevant lists on KION and is level with generation on Yambda. Credit along the list and look-ahead help only rewards that depend on what follows an item.

5.5 Q5. What is the trade-off between relevance and reward?

How RL settings and the reward shape it. Every RL setting we varied moves the policy along one curve: the anchor weight, the KL budget, the estimator, the credit assignment, the simulated responses and the capacity (Figure 4, Section 5.4). **Among RL settings, only the reward moves the frontier; the others choose a point on it.** Equation 2 says why for per-item rewards: the settings change how far the policy moves towards π^*, while only r changes where π^* lies. Rank-1 LoRA lies on the frontier. GRPO is the one setting that may lie slightly above it on KION: at equal KL it reaches 7% to 10% more predicted share than the value model, at no lower NDCG@10. On Yambda it trades along the frontier (76% of the value model's gain at a higher NDCG@10; seed 0).

Can the reward rise without a noticeable loss of relevance? Table 11 in the appendix compares methods at equal relevance: the largest gain each reaches while staying within 1% or 5% of the base recommender's NDCG@10. Between two runs of a method we draw a straight line, which a random mixture of the two policies achieves. On KION, RL on the share gains 4.2% and 14.1% of predicted share at the two levels, and 1.8% and 3.7% of logged share. Adding ten times the logged relevance to the reward moves the frontier: 16.9% and 23.1% of predicted share. Re-ranking by the same reward model reaches 14.9% and 20.7%, with the largest logged gain (15.6%). Most of that logged gain comes from where re-ranking starts: the base recommender's ranked lists alone already have an 11.7% higher logged share than its generated lists. On Yambda the differences at 1% are within the noise between seeds, and the methods that learn from the log lead in logged share through repeats.

Does it take RL? No. Re-ranking with weight w computes the top of the anchored optimum for $\beta = w$. At the main run's predicted share, re-ranking is more relevant (NDCG@10 0.1119 ± 0.0013 against 0.1041 ± 0.0020) and has a higher logged share (0.0201 ± 0.0001 against 0.0176 ± 0.0002). It is also more personal: $8.9\% \pm 0.3$ of its gain is personal, against 0.2% of RL's (Table 17). Most of the relevance gap is a head start. Ranked

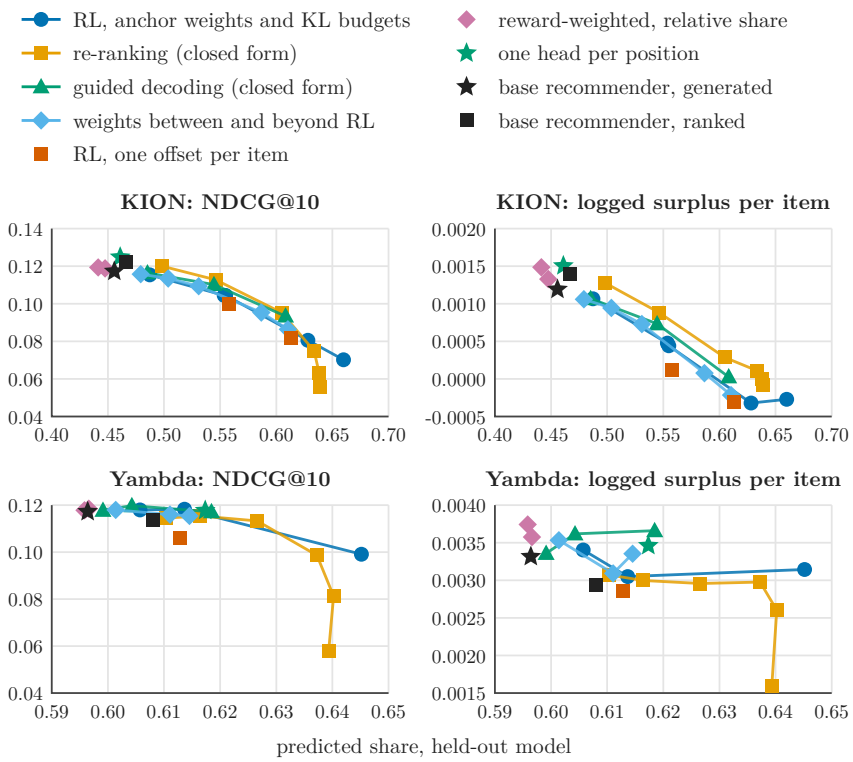


Figure 6: Methods against the predicted share (frontier sessions, seed 0). RL, its interpolated and extrapolated weights, re-ranking and guided decoding share one frontier; reward-weighted fine-tuning on the relative share and one head per position leave it towards relevance and a higher logged surplus. Re-ranking starts from the ranked lists, the other methods from the generated ones.

lists are 4% more relevant than generated ones on KION, and from its own start each method pays about the same relevance for the same predicted share.

Does it take training? Not to reach the frontier (Figure 6). Interpolating the weights between the base recommender and the main run gives policies on the anchor sweep's curve, or slightly above it at equal predicted share (NDCG@10 $+0.0001 \pm 0.0002$ to $+0.0009 \pm 0.0002$). Extrapolating to twice the change stays on it (-0.0009 ± 0.0014). On Yambda, whose main run barely moved, extrapolation falls 0.003 below the run at $\beta = 0.1$. Guided decoding beats RL at equal relevance on both datasets. On Yambda at strength 0.3 it raises NDCG@10 by 2.1% and the logged surplus by 9%, at about RL's predicted share.

Rewards that move the frontier. On KION, the share plus ten times logged relevance at $\beta = 0.3$ beats the base recommender's generated lists on three counts at once: relevance (NDCG@10 0.1202 ± 0.0007 against 0.1177 ± 0.0008), predicted share (+10%) and logged share (+7%). It does not reach the relevance of the base recommender's ranked lists (0.1221 ± 0.0004). RL on the relative share removes the drift towards films and keeps relevance, but gains almost nothing (Table 18). Re-ranking by the relative share does better against the ranked lists it starts from. At $w = 0.3$ it raises NDCG@10 from 0.1221 ± 0.0004 to 0.1234 ± 0.0006 and the logged surplus by 15%. At $w = 0.1$ it raises the surplus by 38% for 0.5% of NDCG@10, in every seed (Table 10). On Yambda, logged relevance alone raises NDCG@10 by 23%, but by replaying histories: relevance on new tracks falls from 0.0260 to 0.0198.

Learning from the log. Every method above reads the reward model. Reading the log instead does better on the outcomes that the reward model cannot inflate. On KION, reward-weighted fine-tuning on the logged relative share ($\tau = 0.3$) raises NDCG@10 by $1.9\% \pm 0.5$, the logged surplus by $28\% \pm 3$, the personal part by $19\% \pm 1$ and coverage by $5\% \pm 2$ (Table 8). For NDCG@10 and the surplus, the per-user intervals exclude zero in every seed. It lowers the predicted share by 3.8%: it moves towards what a user takes to more than usual, not towards what the reward model rates.

On the plain consumed share, the same fine-tuning reproduces RL's drift without any reward model. The predicted share rises by $20.8\% \pm 0.5$ (RL: $24.0\% \pm 2.0$), NDCG@10 falls by $13.4\% \pm 0.3$, the logged surplus by 75%, and the recommended titles are half as long. The drift therefore lies in the definition of the share, not in RL. Within-user DPO buys a larger surplus ($+42\% \pm 5$) and personal part ($+35\% \pm 6$), but pays in relevance ($-3.7\% \pm 1.6$) and coverage ($-22\% \pm 2$). On Yambda, reward-weighted fine-tuning raises the surplus

Table 8: Learning from logged responses alone, without a reward model (frontier sessions): change against the base recommender's generated lists, in percent. "Logged surplus": the logged response to recommended items relative to what is typical for each item; "Personal part": the held-out reward model's prediction for the user minus that for another user.

	NDCG@10	Logged surplus	Personal part	Coverage	Pred. share
KION, 3 seeds					
RL, main run ($\beta = 0.3$)	-11.6 ± 1.5	-60.4 ± 3.7	$+2.2 \pm 2.9$	-13.0 ± 3.2	$+24.0 \pm 2.0$
Liked-only fine-tuning	-11.7 ± 0.3	-71.6 ± 3.5	$+2.2 \pm 1.5$	-4.8 ± 0.9	$+20.3 \pm 0.5$
Reward-weighted, relative share, $\tau = 0.3$	$+1.9 \pm 0.5$	$+27.7 \pm 2.8$	$+18.7 \pm 0.9$	$+5.3 \pm 1.7$	-3.8 ± 1.0
Reward-weighted, relative share, $\tau = 1$	$+1.3 \pm 0.1$	$+12.9 \pm 1.5$	$+6.9 \pm 0.9$	$+9.0 \pm 1.6$	-2.3 ± 0.8
Reward-weighted, consumed share, $\tau = 0.3$	-13.4 ± 0.3	-74.7 ± 0.9	$+0.5 \pm 2.2$	-3.4 ± 0.9	$+20.8 \pm 0.5$
Within-user DPO, $\beta = 0.5$	-3.7 ± 1.6	$+42.0 \pm 4.9$	$+35.1 \pm 5.5$	-22.3 ± 2.0	-8.0 ± 0.8
Within-user DPO, $\beta = 0.1$	-18.7 ± 2.9	$+1.1 \pm 6.3$	$+9.2 \pm 9.2$	-57.9 ± 2.4	-10.8 ± 3.7
Yambda, 1 seed					
RL, main run ($\beta = 0.3$)	$+0.6$	$+2.7$	$+1.4$	-2.7	$+1.6$
Liked-only fine-tuning	$+2.2$	$+7.9$	$+3.4$	-0.9	$+1.1$
Reward-weighted, relative share, $\tau = 0.3$	$+0.5$	$+12.9$	$+1.4$	-1.1	-0.1
Reward-weighted, relative share, $\tau = 1$	$+1.1$	$+7.9$	$+0.8$	-0.2	$+0.0$
Within-user DPO, $\beta = 0.1$	-24.2	$+32.3$	$+9.3$	-36.7	-0.6

by 13% at unchanged relevance, and liked-only fine-tuning, its hard-threshold form, is more relevant still (NDCG@10 +2.2%).

A control for the extra epoch. Reward-weighted fine-tuning gives π_0 one more epoch of training at a tenth of its learning rate. Two effects other than the weights could explain its gain. First, π_0 had stopped while its learning rate was still high, so any extra epoch might help. Second, on KION ranked lists are 4% more relevant than generated ones anyway. A control session with seeds 0 to 2 separates these effects, with its hypotheses written down before the run (Table 9).

The same epoch with every weight equal to one, *continued training*, does help. It lowers the next-item loss on the validation period in every seed. It raises coverage by 10%, NDCG@10 of generated lists by 0.7% (significant in two seeds of three) and the logged surplus by 5% (in one seed of three). The weights add more on top of it, in every seed: $21\% \pm 4$ of logged surplus, $19\% \pm 2$ of personal part and $1.3\% \pm 0.5$ of NDCG@10 on generated lists, and 17%, 15% and 0.7% on ranked lists. Against continued training, coverage falls by 3% to 4%. The fine-tuned recommender's ranked lists are also $0.9\% \pm 0.3$ more relevant than those of π_0, with 23% more logged surplus, in every seed. In sum, the extra epoch gives a third of the relevance gain and all of the coverage gain. The weights give the other two thirds of the relevance gain and almost all of the gain in logged surplus and personalisation.

Answer to Q5. Among RL settings, only the reward moves the frontier; the others choose a point on it, and re-ranking, guided decoding and weight interpolation reach that point without RL training. The frontier moves when training draws on the log. Logged relevance in the reward moves it, and reward-weighted fine-tuning on the logged relative share moves it most. Without a reward model, this fine-tuning raises the logged surplus by 17–21% and personalisation by 15–19% over the same epoch of plain training, with relevance up, on generated and on ranked lists.

Table 9: Learning from the log against the same epoch of training without its weights (KION, control session, seeds 0 to 2): change in percent, mean ± standard deviation over the seeds, each change computed within its seed and against lists of the same kind. "Continued training" weighs every logged item one; "Surplus": the logged surplus; "Personal": the personal part of the predicted share.

	Generated lists				Ranked lists			
	NDCG@10	Surplus	Personal	Coverage	NDCG@10	Surplus	Personal	Coverage
Against the base recommender's lists of the same kind								
Continued training	$+0.7 \pm 0.3$	$+5.3 \pm 2.3$	-0.4 ± 1.2	$+10.2 \pm 1.4$	$+0.2 \pm 0.3$	$+5.1 \pm 4.0$	-1.0 ± 2.1	$+8.8 \pm 1.2$
Reward-weighted, $\tau = 1$	$+1.3 \pm 0.1$	$+12.9 \pm 1.5$	$+6.9 \pm 0.9$	$+9.0 \pm 1.6$	$+0.6 \pm 0.4$	$+11.6 \pm 4.0$	$+4.5 \pm 1.3$	$+8.1 \pm 1.1$
Reward-weighted, $\tau = 0.3$	$+1.9 \pm 0.5$	$+27.7 \pm 2.8$	$+18.7 \pm 1.0$	$+5.3 \pm 1.7$	$+0.9 \pm 0.3$	$+22.7 \pm 4.6$	$+13.4 \pm 0.5$	$+5.4 \pm 1.1$
Within-user DPO, $\beta = 0.5$	-3.7 ± 1.6	$+42.0 \pm 4.9$	$+35.2 \pm 5.6$	-22.3 ± 2.0	-3.6 ± 0.4	$+40.6 \pm 1.1$	$+30.1 \pm 2.6$	-11.0 ± 1.0
Against continued training, lists of the same kind								
Reward-weighted, $\tau = 1$	$+0.6 \pm 0.3$	$+7.2 \pm 1.5$	$+7.3 \pm 0.8$	-1.1 ± 0.3	$+0.4 \pm 0.1$	$+6.2 \pm 0.7$	$+5.7 \pm 1.0$	-0.7 ± 0.1
Reward-weighted, $\tau = 0.3$	$+1.3 \pm 0.5$	$+21.3 \pm 3.7$	$+19.2 \pm 2.0$	-4.4 ± 0.5	$+0.7 \pm 0.1$	$+16.8 \pm 0.1$	$+14.6 \pm 2.2$	-3.2 ± 0.1
Within-user DPO, $\beta = 0.5$	-4.3 ± 1.6	$+35.0 \pm 6.9$	$+35.8 \pm 7.1$	-29.5 ± 1.7	-3.8 ± 0.1	$+33.9 \pm 4.1$	$+31.6 \pm 5.4$	-18.2 ± 1.3

6 Discussion

What a second reward model catches. A held-out reward model, the usual defence against reward hacking, catches exploitation of one particular set of weights. Without an anchor, the training model predicts a consumed share of 0.84 for its policy's lists, but the held-out model only 0.69 (Table 15). It misses what any model of this kind invites, as Eisenstein et al. [22] found for ensembles. Both of our models learned, correctly, that some titles are finished more often than others. And both extrapolate the return after the list beyond any rate in the log.

Cheap checks that do catch it. Three instruments expose this at little cost: scoring the same lists for another user, splitting the policy's change into a per-item part and the rest, and counting the logged response to recommended items. One quantity even predicts it before any RL: the per-item share of the reward's predictions says how much of a policy's change will be one re-weighting of the catalogue.

What moved the frontier. For a reward that sums per-item predictions, the anchored optimum can be reached without a policy gradient (Equation 2). What moved the frontier came from two other places. One was the log, through a reward that compares a response with what is typical for the item. The other was the list format, which should follow how strongly one consumption predicts the next.

In practice:

- compute the reward's per-item share before fine-tuning;
- report the KL distance a run ended at, not its anchor weight;
- re-rank or decode by the reward model before training against it;
- fit the anchored optimum of a length-neutral logged reward directly, by reward-weighted fine-tuning;
- choose the list format (generated, ranked or one head per position) on validation.

6.1 An online experiment

Offline evidence settles the collapse, the role of the anchor and the impersonality of the predicted gain. Whether users consume more of the fine-tuned lists, return more often or adopt more needs an experiment that serves them. Its arms are set by KL distance, not by anchor weight: on Yambda one grid step separates a working weight from a collapsed policy, so a fixed weight does not fix a distance.

- *Arms:* (A) the base recommender; (B) RL held at the KL distance of the validation-selected run (0.015 nats per item on KION, 0.29 on Yambda); (C) re-ranking by the reward model at the predicted share of B; (D) reward-weighted fine-tuning on the relative share, $\tau = 0.3$; (E) one head per position; (F) on the music service, decoding guided by adoption with look-ahead at strength 0.3.
- *Primary metrics, per user, over four weeks of hashed assignment* (one week for exposure to settle, fourteen days of adoption window, one more week for returns): the consumed share of served recommendations and its surplus over what is typical for each item, the return rate, and on Yambda the share of first plays of recommended tracks played again within 14 days.
- *Guardrails:* consumption per user, likes and dislikes, coverage and Gini of impressions, the share of new items, and every metric for the quartile of users with the shortest histories.
- *Analysis and size:* per-user differences with a CUPED adjustment on the four weeks before, bootstrap intervals, and history quartiles as pre-registered segments. A 1% relative change in a binary outcome near 0.7 needs about $2(1.96 + 0.84)^2 \cdot 0.7 \cdot 0.3/0.007^2 \approx 67,000$ users per arm ($\alpha = 0.05$, power 0.8).

7 Limitations, Future Work and Conclusion

Limitations. All results are offline, and the logged check sees only the recommended items that users reached on their own. Yambda's frontier session, the long-term rewards and the analyses by position, user and item have one seed, and adoption could not be verified on a test week. The models are small (4 layers of width 256), so a larger reward model may know more about users. Yambda's models read 118 events of a median 2,638, its grids are narrower than KION's, and the baselines are untuned. Guided decoding calls the reward model for 50 candidates at every position, the heads per position sit on a frozen recommender, and reward-weighted fine-tuning and DPO ran at one learning rate. Early stopping kept the base recommender while its learning rate was still high. One more epoch of plain training lowers its validation loss and raises NDCG@10 of its generated lists by 0.7%, so every change measured against the base recommender includes this slack. It is small next to RL's loss, and a third of the relevance gained by learning from the log. Finally, the equal-relevance curves interpolate between runs, the per-item share is measured on 100 candidates, and the personal part is the reward model's own belief.

Future work. The online experiment of Section 6.1 is the decisive next step. Offline, we see six directions:

- a base recommender trained until its learning rate has decayed, as the reference for every method;
- the per-user counterpart of the relative share from OneRec-V2 [43] as the logged reward of reward-weighted fine-tuning;
- heads per position trained together with the recommender and combined with reward-weighted fine-tuning;
- guided decoding with the value of the optimum rather than of π_0, as Q♯ iterates it [34];
- a larger reward model, whose per-item share would say in advance whether RL can learn more than a re-weighting of the catalogue;
- a longer Yambda context, and a test period long enough to observe adoption.

Conclusion. We fine-tuned a generative recommender against a learned reward model, with only a KL anchor to keep it relevant. The predicted reward rose, and a reward model trained on other users confirmed the rise. But what RL had learned was largely one re-weighting of the catalogue towards the items finished most often. Measured against what is typical for each item, the logged response to the recommendations fell. Relevance was traded along a frontier that RL settings only move along and that inference reaches without training. What moved the frontier came from the log and from the list format. Reward-weighted fine-tuning on the consumed share relative to what is typical for the item improved the logged surplus, personalisation and relevance beyond the same epoch of plain training, on generated and on ranked lists. Lists predicted position by position from the real history were more relevant than generated ones. A reward model is worth optimising with RL only if it knows something about the user that cannot be looked up per item, and its per-item share shows whether it does before any training.

Code and runs. Code, configurations and every table are public at `github.com/daniilantonovv/RecSysMaxxing`. The runs were executed in the Kaggle notebook `kaggle.com/code/daniilantonov5/gptrec-rtx-pro-6000`, whose published versions keep their outputs.

References

[1] W.-C. Kang and J. McAuley, "Self-Attentive Sequential Recommendation," *arXiv preprint arXiv:1808.09781*, 2018.

[2] A. V. Petrov and C. Macdonald, "Generative Sequential Recommendation with GPTRec," *arXiv preprint arXiv:2306.11114*, 2023.

[3] S. Rajput *et al.*, "Recommender Systems with Generative Retrieval," *arXiv preprint arXiv:2305.05065*, 2023.

[4] J. Zhai *et al.*, "Actions Speak Louder than Words: Trillion-Parameter Sequential Transducers for Generative Recommendations," *arXiv preprint arXiv:2402.17152*, 2024.

[5] J. Deng *et al.*, "OneRec: Unifying Retrieve and Rank with Generative Recommender and Iterative Preference Alignment," *arXiv preprint arXiv:2502.18965*, 2025.

[6] M. Chen, A. Beutel, P. Covington, S. Jain, F. Belletti, and E. Chi, "Top-K Off-Policy Correction for a REINFORCE Recommender System," *arXiv preprint arXiv:1812.02353*, 2018.

[7] E. Ie *et al.*, "Reinforcement Learning for Slate-based Recommender Systems: A Tractable Decomposition and Practical Methodology," *arXiv preprint arXiv:1905.12767*, 2019.

[8] A. Petrov and C. Macdonald, "Aligning GPTRec with Beyond-Accuracy Goals with Reinforcement Learning," *arXiv preprint arXiv:2403.04875*, 2024.

[9] A. Matveev, S. Makeev, A. Krasilnikov, V. Baikalov, S. Liamaev, and K. Khrylchenko, "Session-Level Optimization for Large-Scale Retrieval using REINFORCE with Multi-Step Off-Policy Correction," *arXiv preprint arXiv:2607.02818*, 2026.

[10] L. Gao, J. Schulman, and J. Hilton, "Scaling Laws for Reward Model Overoptimization," *arXiv preprint arXiv:2210.10760*, 2022.

[11] J. Skalse, N. H. R. Howe, D. Krasheninnikov, and D. Krueger, "Defining and Characterizing Reward Hacking," *arXiv preprint arXiv:2209.13085*, 2022.

[12] C. Gao *et al.*, "Alleviating Matthew Effect of Offline Reinforcement Learning in Interactive Recommendation," *arXiv preprint arXiv:2307.04571*, 2023.

[13] R. Jiang, S. Chiappa, T. Lattimore, A. György, and P. Kohli, "Degenerate Feedback Loops in Recommender Systems," *arXiv preprint arXiv:1902.10730*, 2019.

[14] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever, "Language Models are Unsupervised Multitask Learners," technical report, 2019.

[15] B. Hidasi, A. Karatzoglou, L. Baltrunas, and D. Tikk, "Session-based Recommendations with Recurrent Neural Networks," *arXiv preprint arXiv:1511.06939*, 2015.

[16] F. Sun *et al.*, "BERT4Rec: Sequential Recommendation with Bidirectional Encoder Representations from Transformer," *arXiv preprint arXiv:1904.06690*, 2019.

[17] X. Xin, A. Karatzoglou, I. Arapakis, and J. M. Jose, "Self-Supervised Reinforcement Learning for Recommender Systems," *arXiv preprint arXiv:2006.05779*, 2020.

[18] E. Ie *et al.*, "RecSim: A Configurable Simulation Platform for Recommender Systems," *arXiv preprint arXiv:1909.04847*, 2019.

[19] K. Zhao *et al.*, "KuaiSim: A Comprehensive Simulator for Recommender Systems," *arXiv preprint arXiv:2309.12645*, 2023.

[20] T. Yu *et al.*, "MOPO: Model-based Offline Policy Optimization," *arXiv preprint arXiv:2005.13239*, 2020.

[21] R. Kidambi, A. Rajeswaran, P. Netrapalli, and T. Joachims, "MOReL: Model-Based Offline Reinforcement Learning," *arXiv preprint arXiv:2005.05951*, 2020.

[22] J. Eisenstein *et al.*, "Helping or Herding? Reward Model Ensembles Mitigate but do not Eliminate Reward Hacking," in *Conference on Language Modeling*, 2024.

[23] N. Jaques *et al.*, "Way Off-Policy Batch Deep Reinforcement Learning of Implicit Human Preferences in Dialog," *arXiv preprint arXiv:1907.00456*, 2019.

[24] D. M. Ziegler *et al.*, "Fine-Tuning Language Models from Human Preferences," *arXiv preprint arXiv:1909.08593*, 2019.

[25] L. Ouyang *et al.*, "Training language models to follow instructions with human feedback," *arXiv preprint arXiv:2203.02155*, 2022.

[26] R. Zhan *et al.*, "Deconfounding Duration Bias in Watch-time Prediction for Video Recommendation," *arXiv preprint arXiv:2206.06003*, 2022.

[27] H. Zhao, L. Zhang, J. Xu, G. Cai, Z. Dong, and J.-R. Wen, "Uncovering User Interest from Biased and Noised Watch Time in Video Recommendation," *arXiv preprint arXiv:2308.08120*, 2023.

[28] L. Zou, L. Xia, Z. Ding, J. Song, W. Liu, and D. Yin, "Reinforcement Learning to Optimize Long-term User Engagement in Recommender Systems," *arXiv preprint arXiv:1902.05570*, 2019.

[29] Q. Cai *et al.*, "Reinforcing User Retention in a Billion Scale Short Video Recommender System," *arXiv preprint arXiv:2302.01724*, 2023.

[30] J. Carbonell and J. Goldstein, "The Use of MMR, Diversity-Based Reranking for Reordering Documents and Producing Summaries," in *Proceedings of the 21st Annual International ACM SIGIR Conference on Research and Development in Information Retrieval*, 1998, pp. 335–336.

[31] E. J. Hu *et al.*, "LoRA: Low-Rank Adaptation of Large Language Models," in *International Conference on Learning Representations*, 2022.

[32] J. Schulman and Thinking Machines Lab, "LoRA Without Regret," *Thinking Machines Lab: Connectionism*, 2025, [Online]. Available: https://thinkingmachines.ai/blog/lora/

[33] S. Mudgal *et al.*, "Controlled Decoding from Language Models," in *Proceedings of the 41st International Conference on Machine Learning*, 2024.

[34] J. P. Zhou *et al.*, "Q♯: Provably Optimal Distributional RL for LLM Post-Training," in *Advances in Neural Information Processing Systems*, 2025.

[35] X. B. Peng, A. Kumar, G. Zhang, and S. Levine, "Advantage-Weighted Regression: Simple and Scalable Off-Policy Reinforcement Learning," *arXiv preprint arXiv:1910.00177*, 2019.

[36] K. Chidambaram, S. K. Krishnamurthy, Q. Xu, K.-J. Hsiao, and M. Bhattacharya, "Robust Post-Training for Generative Recommenders: Why Exponential Reward-Weighted SFT Outperforms RLHF," *arXiv preprint arXiv:2603.10279*, 2026.

[37] R. Rafailov, A. Sharma, E. Mitchell, S. Ermon, C. D. Manning, and C. Finn, "Direct Preference Optimization: Your Language Model is Secretly a Reward Model," in *Advances in Neural Information Processing Systems*, 2023.

[38] M. Wortsman *et al.*, "Robust Fine-Tuning of Zero-Shot Models," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2022.

[39] A. Ramé *et al.*, "WARP: On the Benefits of Weight Averaged Rewarded Policies," *arXiv preprint arXiv:2406.16768*, 2024.

[40] C. Zheng, Z. Wang, H. Ji, M. Huang, and N. Peng, "Model Extrapolation Expedites Alignment," in *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics*, 2025.

[41] Z. Shao *et al.*, "DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models," *arXiv preprint arXiv:2402.03300*, 2024.

[42] F. Gloeckle, B. Y. Idrissi, B. Rozière, D. Lopez-Paz, and G. Synnaeve, "Better & Faster Large Language Models via Multi-token Prediction," in *Proceedings of the 41st International Conference on Machine Learning*, 2024.

[43] G. Zhou *et al.*, "OneRec-V2 Technical Report," *arXiv preprint arXiv:2508.20900*, 2025.

[44] J. Schulman, P. Moritz, S. Levine, M. Jordan, and P. Abbeel, "High-Dimensional Continuous Control Using Generalized Advantage Estimation," *arXiv preprint arXiv:1506.02438*, 2015.

[45] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal Policy Optimization Algorithms," *arXiv preprint arXiv:1707.06347*, 2017.

[46] W. Kool, H. van Hoof, and M. Welling, "Buy 4 REINFORCE Samples, Get a Baseline for Free!," in *ICLR 2019 Workshop on Deep Reinforcement Learning Meets Structured Prediction*, 2019.

[47] A. Petrov, I. Safilo, D. Tikhonovich, and D. Ignatov, "MTS Kion Implicit Contextualised Sequential Dataset for Movie Recommendation," *arXiv preprint arXiv:2209.00325*, 2022.

[48] A. Ploshkin *et al.*, "Yambda-5B — A Large-Scale Multi-modal Dataset for Ranking And Retrieval," *arXiv preprint arXiv:2505.22238*, 2025.

A Sessions and seeds

Each dataset ran three sessions on one 96 GB GPU. A *final* session ran the full grid (6.1 hours on KION, 12.3 on Yambda). A *review* session added the long-term rewards, the reward models with hidden responses, and the analyses by position, user and item. A *frontier* session ran the methods that probe what RL learns (4.8 and 6.0 hours); Yambda's ran without likes and dislikes, which none of its rewards use. On KION two more sessions followed. The *control* session ran seeds 0 to 2 itself (20 minutes per seed) to test learning from the log against continued training. The *quick* session repeated the core of the study with seed 0 in 55 minutes and reproduced the final session's numbers.

Seeds 1 and 2 repeat the core on both datasets: the base models, the main run, the anchor sweep, a KL budget, the credit and estimator comparisons, the alternatives to RL and logged relevance. On KION they also repeat the frontier comparisons. A seed fixes the initial weights of every model and the sampling of fine-tuning. Every session retrains every model with the same seed, so a run present in two sessions differs only in the third decimal.

B Supporting tables

Table 10: The central comparisons over three seeds (0, 1, 2): mean ± standard deviation. A seed fixes the initial weights of every model and the sampling of fine-tuning. "Budget" is a KL budget of 0.1 nats on KION and 0.5 on Yambda; "–": not run with three seeds on that dataset; "<0.0001": a spread below the four decimals at which seeds 1 and 2 were saved.

	KION	**Yambda**
Main run (β = 0.3) against the base recommender		
NDCG@10, change in %	−11.5 ± 1.3	+0.8 ± 0.7
Predicted share, held-out model, change in %	+23.91 ± 2.09	+1.67 ± 0.01
Logged share per recommended item, change in %	+1.1 ± 0.4	+1.7 ± 0.7
Logged surplus per recommended item, change in %	−60.6 ± 2.1	−3.8 ± 3.4
Personal share of the predicted gain, %	0.2 ± 0.5	15.9 ± 16.3
KL from the base recommender	0.091 ± 0.005	0.038 ± 0.001
What the anchor weight means		
KL at β = 0.3, value model	0.081 ± 0.004	–
KL at β = 0.3, leave-one-out baseline	0.543 ± 0.010	–
Weight holding the KL budget, value model	0.256 ± 0.004	0.076 ± 0.011
Weight holding the KL budget, leave-one-out	0.752 ± 0.017	–
NDCG@10 at the budget, whole rest of the list	0.1013 ± 0.0026	0.0960 ± 0.0008
NDCG@10 at the budget, own reward only	0.1070 ± 0.0027	0.1076 ± 0.0014
NDCG@10 at the budget, one item per episode	0.1043 ± 0.0024	0.1075 ± 0.0038
NDCG@10 at β = 0.03 (Yambda's cliff)	–	0.0061 ± 0.0043
Alternatives and rewards		
NDCG@10, RL main run	0.1041 ± 0.0020	0.1190 ± 0.0015
NDCG@10, re-ranking w = 0.3	0.1119 ± 0.0013	0.1155 ± 0.0008
Logged share per item, RL main run	0.0176 ± 0.0002	0.0689 ± 0.0008
Logged share per item, re-ranking w = 0.3	0.0201 ± 0.0001	0.0662 ± 0.0007
Personal share of the gain, re-ranking w = 0.3, %	8.9 ± 0.3	54.7 ± 8.8
NDCG@10, off-policy REINFORCE	0.1035 ± 0.0006	0.1256 ± 0.0009
NDCG@10, share + 10 × logged relevance, β = 0.3	0.1202 ± 0.0007	–
NDCG@10, logged relevance alone, β = 0	0.1179 ± 0.0004	0.1453 ± 0.0014
NDCG@10 on new items, logged relevance alone	–	0.0198 ± 0.0005
Validation-selected weight on KION (β = 1)		
NDCG@10 at β = 1	0.1156 ± 0.0011	–
Predicted share at β = 1	0.478 ± 0.020	–
Logged share per item at β = 1	0.0179 ± 0.0001	–
Logged surplus per item at β = 1	0.0010 ± 0.0001	–
KL at β = 1	0.015 ± 0.001	–
Re-ranking by the relative share on KION, against the ranked lists it starts from		
NDCG@10, ranked lists	0.1221 ± 0.0004	–
NDCG@10, re-ranking w = 0.3	0.1234 ± 0.0006	–
NDCG@10, re-ranking w = 0.1	0.1214 ± 0.0019	–
Logged surplus per item, ranked lists	0.00140 ± 0.00007	–
Logged surplus per item, re-ranking w = 0.3	0.00161 ± 0.00007	–
Logged surplus per item, re-ranking w = 0.1	0.00193 ± 0.00009	–
Reference		
NDCG@10, base recommender	0.1177 ± 0.0008	0.1181 ± 0.0010
Predicted share, base recommender	0.442 ± 0.019	0.596 ± 0.005
Logged share per item, base recommender	0.0174 ± 0.0002	0.0677 ± 0.0011
Logged surplus per item, base recommender	0.0012 ± <0.0001	–
NDCG@10 on new items, base recommender	–	0.0260 ± 0.0004
Reward model AUC, held-out copy	0.754 ± 0.001	0.867 ± 0.001

Table 11: Methods at equal relevance (final session): the largest change of the predicted share (held-out model) reached without losing more than 1% or 5% of the base recommender's NDCG@10, and the change of the logged share at that point, in percent. "–": the method does not reach that relevance.

	NDCG@10 at most 1% below the base		NDCG@10 at most 5% below the base	
	Predicted share, %	**Logged share, %**	**Predicted share, %**	**Logged share, %**
KION				
Ranked list of the base recommender	+2.4	+11.7	+2.4	+11.7
RL on the share, anchor sweep	+4.2	+1.8	+14.1	+3.7
RL on share + 10 × logged relevance	+16.9	+8.8	+23.1	+6.7
Re-ranking by the reward model	+14.9	+15.6	+20.7	+15.5
Off-policy REINFORCE	+1.4	+0.3	+6.9	+1.4
Liked-only fine-tuning	+1.7	+0.3	+8.4	+1.4
Yambda				
Ranked list of the base recommender	+0.6	−1.2	+1.9	−3.6
RL on the share, anchor sweep	+2.8	+0.1	+5.0	−2.5
RL on share + 10 × logged relevance	+1.4	+24.1	+1.4	+24.1
Re-ranking by the reward model	–	–	+5.3	−3.8
Off-policy REINFORCE	+1.9	+9.9	+1.9	+9.9
Liked-only fine-tuning	+1.2	+3.4	+1.2	+3.4

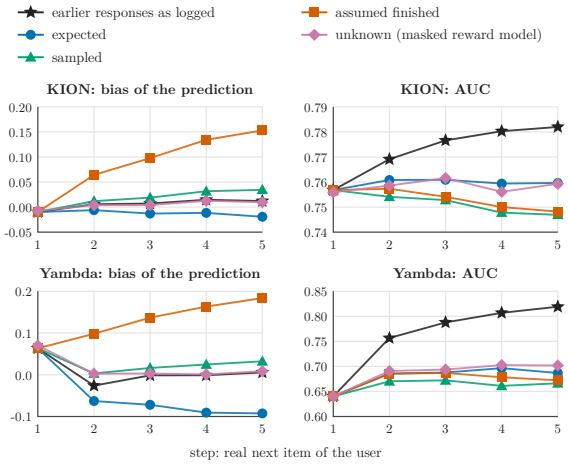


Figure 7: The reward model on the users' real next five items: bias and AUC of the predicted share at each step, with the earlier responses as logged, expected, sampled, finished, or unknown to the model trained with hidden responses (review session).

Table 12: KION: the titles whose number of lists among the 59,997 test requests grew and shrank most under the main run (review session). Titles as given in the English release of the KION item table, which translates the Russian titles; "usual share": the title's mean consumed share in the training period.

	Type	**Hours**	**Viewers in training**	**Usual share**	**Lists before**	**after**
Gained most						
White Snow (2021)	film	2.2	32,435	0.64	8,388	24,472
Palmyra (2020)	film	1.4	18,762	0.54	8,598	21,579
Devyataev (2021)	film	1.9	108,030	0.69	14,471	26,328
Dukhless (2011)	film	1.7	953	0.55	7,308	17,315
Masha (2020)	film	1.5	40,254	0.63	23,485	32,316
Lost most						
Themis sees (2019)	series	16.9	25,571	0.08	33,948	10,781
Listened to (2021)	series	9.5	62,651	0.18	16,278	3,343
The Scam (2021)	series	22.0	51,094	0.11	16,660	4,038
The Germans (2021)	series	10.9	33,452	0.29	14,654	5,371
Flash radio (2019)	film	1.7	8,811	0.33	11,925	5,088

Table 13: NDCG@10 and the change of the logged share per recommended item by the user's number of events before the request, in quartiles (review session, main run).

		NDCG@10				
	Users	**Base, ranked**	**Base, generated**	**RL**	**RL change, %**	**Logged change, %**
KION						
5–8 events	14552	0.1537	0.1515	0.1311	−13.5	−2.4
9–13 events	14121	0.1364	0.1311	0.1171	−10.7	+1.2
14–24 events	15849	0.1151	0.1097	0.0991	−9.7	−0.6
25–1143 events	15475	0.0861	0.0802	0.0743	−7.4	+3.3
Yambda						
50–1076 events	1897	0.1164	0.1147	0.1165	+1.6	+1.3
1077–2960 events	1899	0.1179	0.1236	0.1237	+0.1	+3.1
2961–6420 events	1898	0.1110	0.1158	0.1162	+0.3	+2.4
6421–25401 events	1899	0.1101	0.1150	0.1159	+0.8	+1.5

Table 14: Concentration of impressions: catalogue coverage, Gini coefficient of the impressions per item, and the share of impressions taken by the 1% most shown items (review session).

	Coverage	**Gini of impressions**	**Top 1% of items, % of impressions**
KION			
Base recommender, ranked	0.574	0.949	62
Base recommender, generated	0.495	0.963	72
RL, $\beta = 0.3$ (main run)	0.452	0.969	76
$\beta = 0.1$	0.278	0.990	92
$\beta = 0$	0.008	0.999	100
Yambda			
Base recommender, ranked	0.466	0.789	23
Base recommender, generated	0.519	0.737	18
RL, $\beta = 0.3$ (main run)	0.512	0.747	20
$\beta = 0.05$	0.446	0.808	29

Table 15: Anchor sweep with the value-model estimator, 1,000 iterations per run (final session). KL is in nats per generated item; "Training model" is the share predicted by the model that produced the rewards.

	KL	**NDCG@10**	**on liked**	**Training model**	**Pred. share**	**Logged**	**Coverage**	**Overlap**
KION								
Base recommender	0.000	0.1173	0.1214	0.417	0.455	0.0173	0.495	0.209
$\beta = 1$	0.015	0.1154	0.1245	0.445	0.487	0.0178	0.479	0.219
$\beta = 0.5$	0.040	0.1120	0.1256	0.470	0.517	0.0180	0.458	0.237
$\beta = 0.3$	0.080	0.1043	0.1233	0.500	0.553	0.0175	0.436	0.251
$\beta = 0.2$	0.140	0.0983	0.1193	0.521	0.576	0.0168	0.413	0.266
$\beta = 0.1$	0.590	0.0714	0.0910	0.616	0.655	0.0114	0.287	0.316
$\beta = 0$	6.215	0.0005	0.0008	0.841	0.687	0.0002	0.008	0.983
$\beta = 0.3$, 2,000 iterations	0.092	0.1047	0.1229	0.502	0.553	0.0174	0.449	0.251
Yambda								
Base recommender	0.000	0.1172	0.1005	0.594	0.599	0.0669	0.519	0.001
$\beta = 0.3$	0.029	0.1176	0.1018	0.604	0.608	0.0681	0.505	0.002
$\beta = 0.1$	0.102	0.1158	0.1008	0.615	0.617	0.0668	0.479	0.004
$\beta = 0.05$	0.286	0.1115	0.0980	0.629	0.629	0.0653	0.433	0.006
$\beta = 0.03$	4.391	0.0032	0.0033	0.871	0.774	0.0017	0.015	0.766
$\beta = 0$	6.926	0.0008	0.0009	0.861	0.810	0.0003	0.001	0.999
$\beta = 0.3$, 2,000 iterations	0.037	0.1183	0.1029	0.608	0.609	0.0686	0.506	0.002

Table 16: Baselines on generated or ranked lists of ten (final session); every baseline reads the recommender's window of history. On KION the history's items are blocked, so replay of the history reduces to popularity and the two rows coincide. The last two rows are the recommender that is fine-tuned.

	KION			**Yambda**			
	NDCG@10	**Recall@10**	**Coverage**	**NDCG@10**	**on repeats**	**on new**	**Coverage**
Most popular	0.0903	0.1303	0.004	0.0186	0.0193	0.0115	0.000
Replay of the history	0.0903	0.1303	0.004	0.1873	0.3722	0.0000	0.519
Markov chain	0.0959	0.1394	0.466	0.0550	0.0934	0.0139	0.457
GRU4Rec	0.1215	0.1721	0.544	0.1116	0.1706	0.0298	0.451
SASRec	0.1082	0.1580	0.335	0.0743	0.1070	0.0242	0.367
BERT4Rec	0.1290	0.1811	0.427	0.1036	0.1717	0.0232	0.461
GPT-2, ranked	0.1220	0.1726	0.574	0.1138	0.1765	0.0315	0.466
GPT-2, generated	0.1173	0.1620	0.495	0.1172	0.1865	0.0261	0.519

Table 17: Ways of raising the reward without on-policy RL (final session). Re-ranking starts from the ranked lists of the base recommender, every other method from its generated lists.

	NDCG@10	on liked	above usual	Pred. share	Logged	Surplus	Coverage
KION							
Base recommender	0.1173	0.1214	0.1261	0.455	0.0173	0.0012	0.495
Base recommender, ranked	0.1220	0.1293	0.1326	0.466	0.0193	0.0014	0.574
RL, $\beta = 0.3$ (main run)	0.1047	0.1229	0.1142	0.553	0.0174	0.0005	0.449
Liked-only fine-tuning	0.1036	0.1258	0.1134	0.545	0.0178	0.0003	0.473
Off-policy REINFORCE, $\varkappa = 0.5$	0.1032	0.1237	0.1127	0.531	0.0179	0.0005	0.504
Re-ranking, $w = 1$	0.1201	0.1334	0.1315	0.497	0.0199	0.0013	0.568
Re-ranking, $w = 0.3$	0.1127	0.1344	0.1249	0.545	0.0201	0.0009	0.531
Re-ranking, $w = 0.1$	0.0953	0.1234	0.1080	0.603	0.0186	0.0003	0.434
Best of 8 sampled lists	0.1049	0.1125	0.1128	0.502	0.0178	0.0010	0.499
Yambda							
Base recommender	0.1172	0.1005	0.0998	0.599	0.0669	0.0033	0.519
Base recommender, ranked	0.1138	0.0989	0.0986	0.611	0.0645	0.0029	0.466
RL, $\beta = 0.3$ (main run)	0.1183	0.1029	0.1023	0.609	0.0686	0.0032	0.506
Liked-only fine-tuning	0.1198	0.1035	0.1028	0.607	0.0692	0.0036	0.515
Off-policy REINFORCE, $\varkappa = 0.5$	0.1245	0.1090	0.1083	0.611	0.0735	0.0039	0.475
Re-ranking, $w = 1$	0.1144	0.0998	0.0995	0.613	0.0651	0.0030	0.463
Re-ranking, $w = 0.3$	0.1155	0.1012	0.1008	0.619	0.0659	0.0029	0.456
Re-ranking, $w = 0.1$	0.1134	0.0998	0.0993	0.630	0.0653	0.0028	0.440
Best of 8 sampled lists	0.1110	0.0967	0.0961	0.618	0.0655	0.0030	0.498

Table 18: Rewards that an item cannot earn on its own account, and logged relevance (final session). "Rel. share" is the held-out model's predicted share relative to what is typical for the item; 0.5 is "as usual".

	NDCG@10	on liked	Pred. share	Rel. share	Logged	Surplus	Coverage
KION							
Base recommender	0.1173	0.1214	0.455	0.529	0.0173	0.0012	0.495
Share (the main reward), $\beta = 0.3$	0.1043	0.1233	0.553	0.525	0.0175	0.0005	0.436
Base recommender, ranked	0.1220	0.1293	0.466	0.528	0.0193	0.0014	0.574
Relative share, $\beta = 0.3$	0.1168	0.1190	0.434	0.541	0.0168	0.0013	0.481
Relative share, $\beta = 0.1$	0.1114	0.1111	0.401	0.554	0.0154	0.0014	0.439
Re-ranking by relative share, $w = 0.1$	0.1206	0.1263	0.437	0.546	0.0185	0.0020	0.552
Share + 10 × logged relevance, $\beta = 0.3$	0.1194	0.1303	0.495	0.535	0.0185	0.0011	0.426
Share + 10 × logged relevance, $\beta = 0.1$	0.1158	0.1323	0.536	0.540	0.0188	0.0008	0.345
Logged relevance alone, $\beta = 0$	0.1180	0.1279	0.470	0.553	0.0172	0.0007	0.097
Yambda							
Base recommender	0.1172	0.1005	0.599	0.469	0.0669	0.0033	0.519
Share (the main reward), $\beta = 0.1$	0.1158	0.1008	0.617	0.465	0.0668	0.0029	0.479
Base recommender, ranked	0.1138	0.0989	0.611	0.473	0.0645	0.0029	0.466
Relative share, $\beta = 0.1$	0.1107	0.0960	0.595	0.487	0.0628	0.0040	0.476
Re-ranking by relative share, $w = 0.1$	0.1120	0.0977	0.610	0.486	0.0633	0.0034	0.444
Share + 10 × logged relevance, $\beta = 0.1$	0.1309	0.1139	0.606	0.468	0.0758	0.0035	0.516
Share + 10 × logged relevance, $\beta = 0$	0.1433	0.1245	0.608	0.467	0.0830	0.0037	0.501
Logged relevance alone, $\beta = 0$	0.1437	0.1259	0.604	0.468	0.0836	0.0039	0.486

Table 19: Rewards for continuing the session and for returning, per item (final session). "Continues" and "returns" are the held-out model's predictions; on KION a session is a calendar day.

	NDCG@10	Pred. share	Continues	Returns	Coverage	Overlap
KION						
Base recommender	0.1173	0.455	0.345	0.718	0.495	0.209
Session, $\beta = 0.3$	0.0951	0.438	0.404	0.754	0.567	0.078
Session, $\beta = 0.1$	0.0443	0.297	0.545	0.764	0.526	0.106
Return, $\beta = 0.3$	0.1077	0.459	0.368	0.739	0.521	0.141
Return, $\beta = 0.1$	0.0903	0.466	0.392	0.765	0.544	0.093
Share + return, $\beta = 0.3$	0.1091	0.502	0.355	0.733	0.491	0.183
Share + return, $\beta = 0.1$	0.0943	0.546	0.372	0.756	0.477	0.167
Yambda						
Base recommender	0.1172	0.599	0.947	0.913	0.519	0.001
Return, $\beta = 0.1$	0.1158	0.600	0.947	0.913	0.516	0.002